Practical no 1

Write a program to display multiple LED blinking patterns using the GPIO pins of Raspberry Pi and observe the output for different configurations.

1) Using One LED

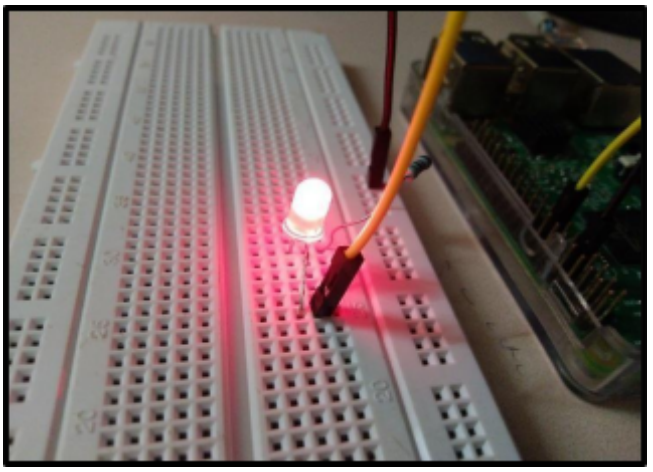
Code:

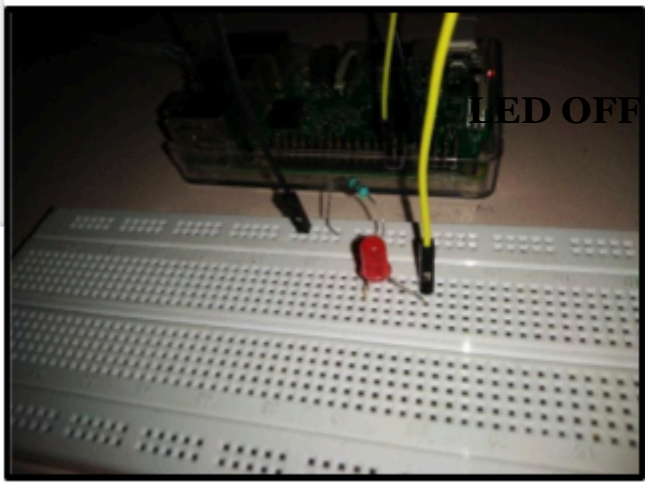
```
import RPI.GPIO as
GPIO import time

GPIO.setmode
(GPIO.BOARD)
GPIO.setup(15,GPIO
.OUT)
for i in range(5):
    GPIO.output(1
    5,True)
    print("ON")
    time.sleep(1)
    GPIO.output(1
    5,False)
    print("OFF")
    time.sleep(1)
print(
"Don
e")
GPIO.
clean
up()
```

Output:

LED ON





LED OFF

1) 3 Led blinking
   Code :

```
import RPI.GPIO as GPIO
import time

GPIO.setmode (GPIO.BOARD)
GPIO.setup(11,GPIO.OUT)
GPIO.setup(13,GPIO.OUT)
GPIO.setup(15,GPIO.OUT)

for i in range(5):
    GPIO.output(11,True) print("ON")
    GPIO.output(11,False)
    print("OFF")

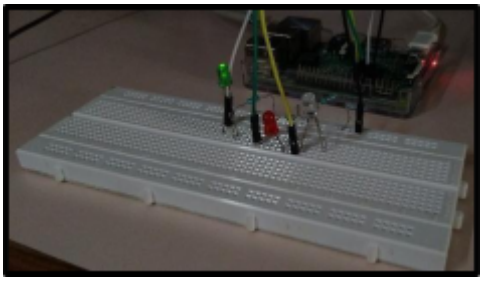
    GPIO.output(13,True) print("ON")
    GPIO.output(13,False)
    print("OFF")

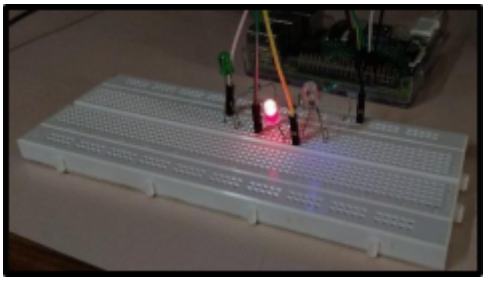
    GPIO.output(15,True) print("ON")
    GPIO.output(15,False)
    print("OFF")

print("Done")
GPIO.cleanup()
```



2) Using Three LED Output:





Practical no 2

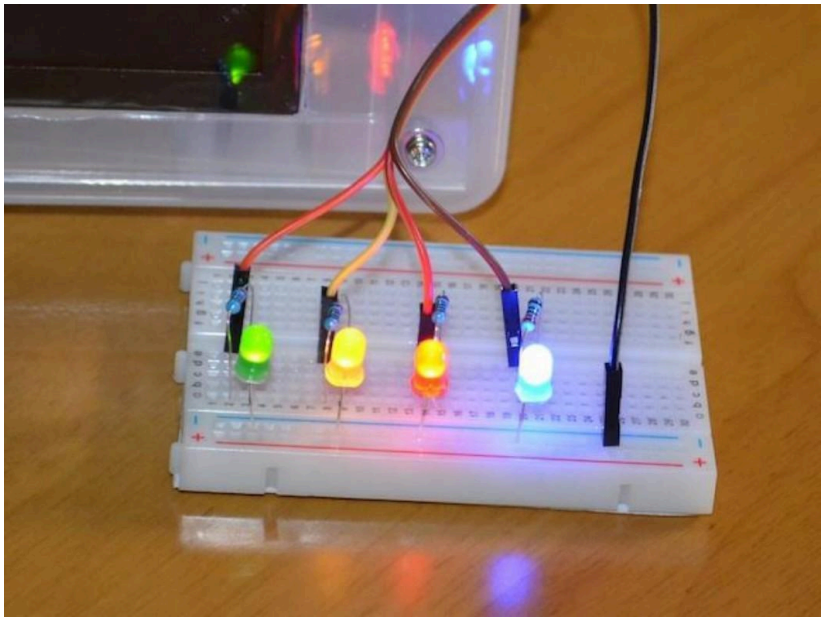
Write a program to display multiple LED blinking patterns using the GPIO pins of Raspberry Pi and observe the output for different configurations.

- **4 Led Blinking Code**

```
import RPI.GPIO as GPIO
```

```
 import time
 GPIO.setmode (GPIO.BOARD)
GPIO.setup(11,GPIO.OUT)
GPIO.setup(13,GPIO.OUT)
GPIO.setup(15,GPIO.OUT)
GPIO.setup(16,GPIO.OUT)
for i in range(5):
GPIO.output(11,True)
print("ON")
GPIO.output(11,False)
print("OFF")
  GPIO.output(13,True)
  print("ON")
  GPIO.output(13,False)
  print("OFF")
  GPIO.output(15,True)
  print("ON")
  GPIO.output(15,False)
  print("OFF")
  GPIO.output(16,True)
  print("ON")
   GPIO.output(16,False)
   print("OFF")
   print("Done")
GPIO.cleanup()
```

output:



Practical no 3
Displaying Time over 4-Digit 7-Segment Display Using Raspberry Pi.

Installation Manual

To display small amount of data with Raspberry Pi, we can use 4 digit 7-segment Display. 7 Segment Display has seven segments in it and each segment has one LED inside it to display the numbers by lighting up the corresponding segments.

Hardware Requirements

1. Raspberry Pi Model A/B/B+
2. 4 digit 7 Segment Display
3. Jumper wires (Female to Female)

Here, I am using **4 digits-7 segments LED** display with TM1637 controller

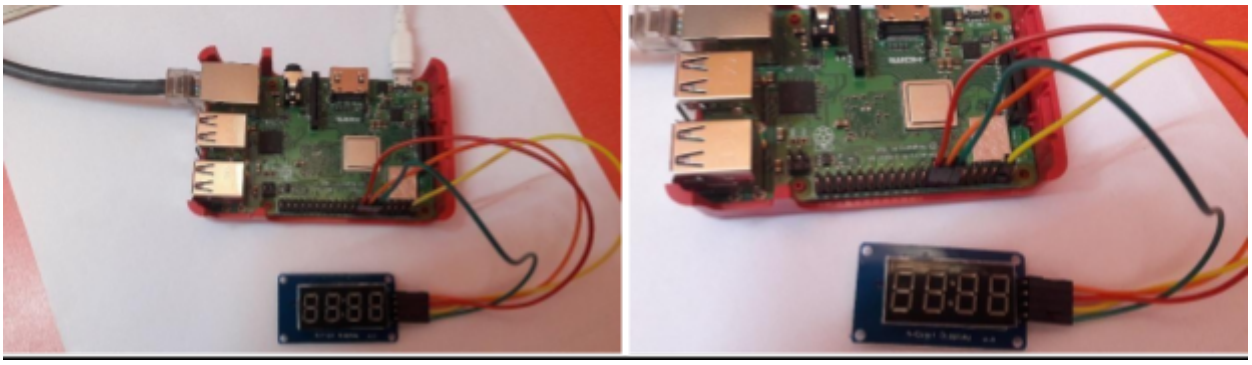




- **Software Requirements**
  1. Raspbian Stretch OS

1. Connect your 4 digit 7 segment display with Raspberry Pi's GPIO Pins.

TM1637 Board Pin	Function	RPI Physical Pin	Raspberry Function
GND	Ground	14	GND
VCC	+ 5V Power	4	5V
DI0	Data In	18	GPIO 24
CLK	Clock	16	GPIO 23



Step 1: Download Python Script

In order to control the LED, using a special script with pre-defined functions. Various functions are available in the script, for example, to display numbers and adjust the intensity of the LEDs.

Create a folder 4digitTime under /home/pi.



Download the script using git command.

https://github.com/tyitraspberrypi/7segment.git

Note: This Script file contains some of the important functions, which are required to add in our Python script.



Step 2: Write Python Script to display Time (e.g clock.py)

```
import sys
import time
import datetime
import RPi.GPIO as GPIO
import tm1637

#CLK -> GPIO23 (Pin 16)
#Di0 -> GPIO24 (Pin 18)
```

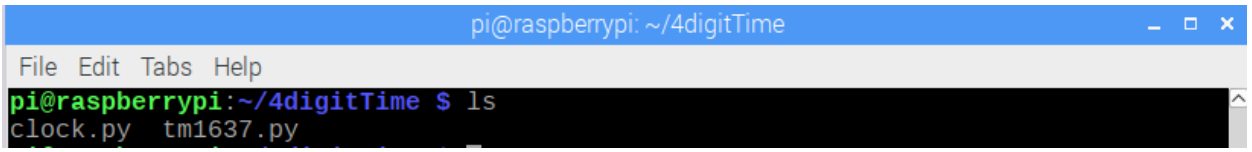
```
Display = tm1637.TM1637(23,24,tm1637.BRIGHT_TYPICAL)

Display.Clear()
Display.SetBrightnes(1)
while(True):
    now =
    datetime.datetime.now() hour
    = now.hour
    minute = now.minute
    second = now.second
    currenttime = [ int(hour / 10), hour % 10, int(minute / 10), minute % 10 ]

    Display.Show(currenttime)
    Display.ShowDoublepoint(second % 2)

    time.sleep(1)
```

The above script needs the **tm1637.py** script to work, so place both files in the same folder.



Script functions

The clock script uses the following functions, defined in tm1637.py:

Display. Clear () - Clears the display if individual LEDs are still active.

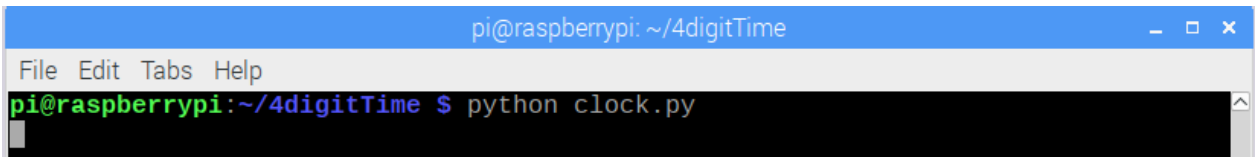
Display.SetBrightnes(x) - After this you can adjust the brightness of the display, at least 0 and maximum 7.

Display. Show(x,x,x,x) - Show the actual 4 digits (digits), x can be 0 to 9.

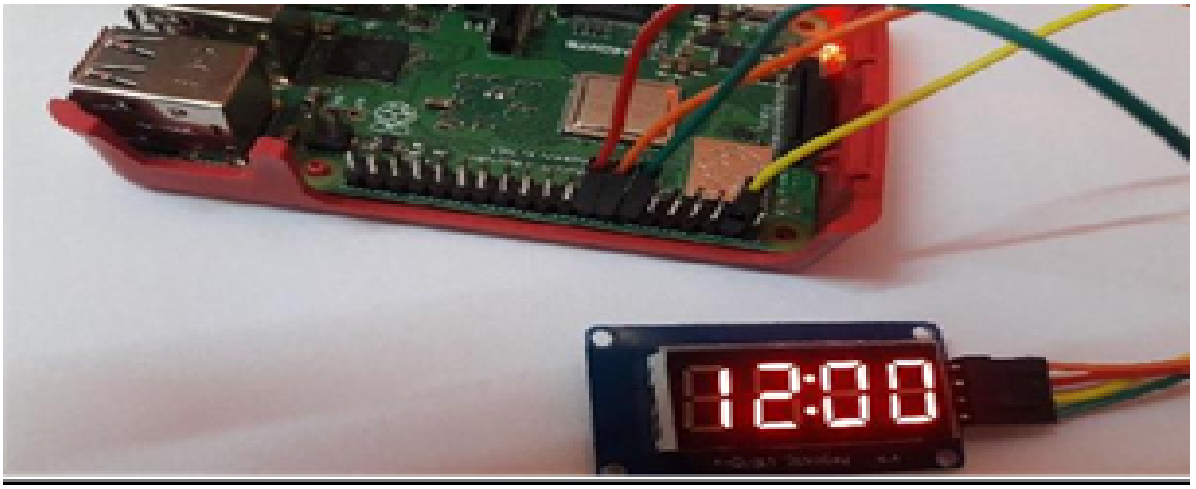
Display.ShowDoublepoint (status) - Controlling the ':' between the second and third digit, true (1) = on / false (0) = off.

To know more about TM1637 controller, check
http://www.microcontroller.it/english/Tutorials/Elettronica/componenti/TM1637.htm

Step 3: Start the script with following command



```
pi@raspberrypi: ~/4digitTime
File  Edit  Tabs  Help
pi@raspberrypi:~/4digitTime $ python clock.py &
[1] 1232
pi@raspberrypi:~/4digitTime $ jobs -l
[1]+  1232 Running                 python clock.py &
pi@raspberrypi:~/4digitTime $
```



Practical no 4
Raspberry Pi Based Oscilloscope

Installation Manual

One of the most important tools in Electrical/Electronic engineering is **The Oscilloscope**.

An oscilloscope is a laboratory instrument commonly used to display and analyze the waveform of electronic signals. In effect, the device draws a graph of the instantaneous signal voltage as a function of time.

In this project we will seek to replicate the signal visualization capabilities of the oscilloscope using the Raspberry Pi and an analog to digital converter module.

Replicating the signal visualization of the oscilloscope using the Raspberry Pi will require the following steps;

1. Perform Digital to analog conversion of the Input signal
2. Prepare the resulting data for representation
3. Plot the data on a live time graph

Analog Signals ⇨ **Signal Processing** ⇨ **Visualize using the Graph**

- **Hardware Requirements**

    1. Raspberry Pi Model A/B/B+
    2. ADS1115 ADC
    3. Breadboard
    4. Jumper Wires

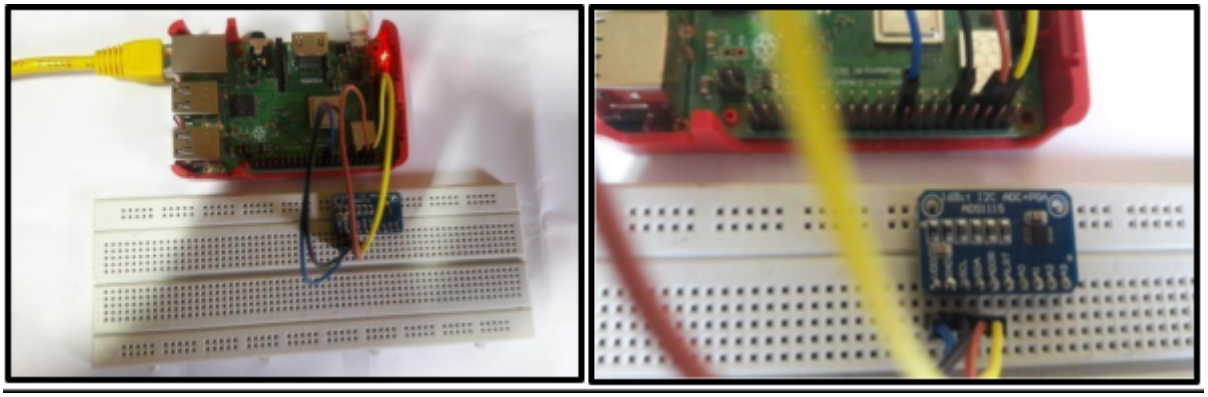
ADS1115 ADC chip is used to convert the analog input signals to digital signals which can be visualized with the Raspberry Pi. This chip is important because the Raspberry Pi does not have an on- board analog to digital converter (ADC).

- **Software Requirements**

    1. Raspbian Stretch OS
    2. Adafruit module for interfacing with the ADS1115 ADC chip
    3. Python Module **matplotlib** used for data visualization

---

ADS1115 ADC	Pin Number	GPIO Number
VDD	Pin 17	3.3v
GND	Pin 9	GND
SCL	Pin 5	GPIO 3
SDA	Pin 3	GPIO 2



$ sudo raspi-config

When the configuration panels open, select interface options, select I2C and click enable.

Step 2: Update the Raspberry pi

The first thing I do before starting any project is updating the Pi. Through this, I am sure every thing on the OS is up to date and I won't experience compatibility issue with any latest software I choose to install on the Pi. To do this, run below two commands:

$ sudo apt-get update
$ sudo apt-get upgrade

Step 3: Install the Adafruit ADS1115 library for ADC

With the update done, we are now ready to install the dependencies starting with the Adafruit python module for the ADS115 chip. Ensure you are in the Raspberry Pi home
directory by running;

$ cd ~
then install the build-essentials by running;

sudo apt-get install build-essential python3-dev python3-smbus git

Next, clone the Adafruit git folder for the library by running;

$ git clone https://github.com/adafruit/Adafruit_Python_ADS1x15.git

Change into the cloned file's directory and run the setup file;
cd Adafruit_Python_ADS1x1z

$ sudo python3 setup.py install

Step 4: Test the library and 12C communication.
Before we proceed with the rest of the project, it is important to test the library and ensure the ADC can communicate with the raspberry pi over I2C. To do this we will use an example script that comes with the library.
While still in the Adafruit_Python_ADS1x15 folder, change directory to the examples
directory by running;
$ cd examples
Next, run the sampletest.py example which displays the value of the four channels on
the ADC in a tabular form.
Run the example using:

```
import time
import board
import busio
import adafruit_ads1x15.ads1015 as ADS
from adafruit_ads1x15.analog_in import AnalogIn

# Create the I2C bus
i2c = busio.I2C(board.SCL, board.SDA)



# Create the ADC object using the I2C bus
ads = ADS.ADS1015(i2c)
```

```
# Create single-ended input on channel 0
chan = AnalogIn(ads, ADS.P0)

# Create differential input between channel 0 and 1
# chan = AnalogIn(ads, ADS.P0, ADS.P1)
print("{:>5}\t{:>5}".format("raw",

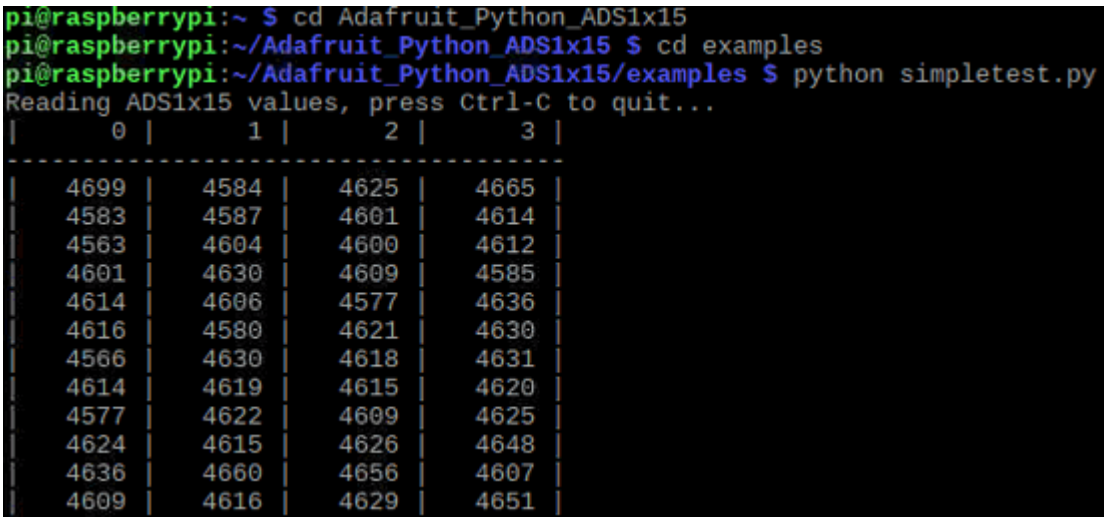
"v"))



while True:

    print("{:>5}\t{:>5.3f}".format(chan.value, chan.
voltage))
    time.sleep(0.5)
```

If the I2C module is enabled and connections good, you should see the data as shown in the image below.



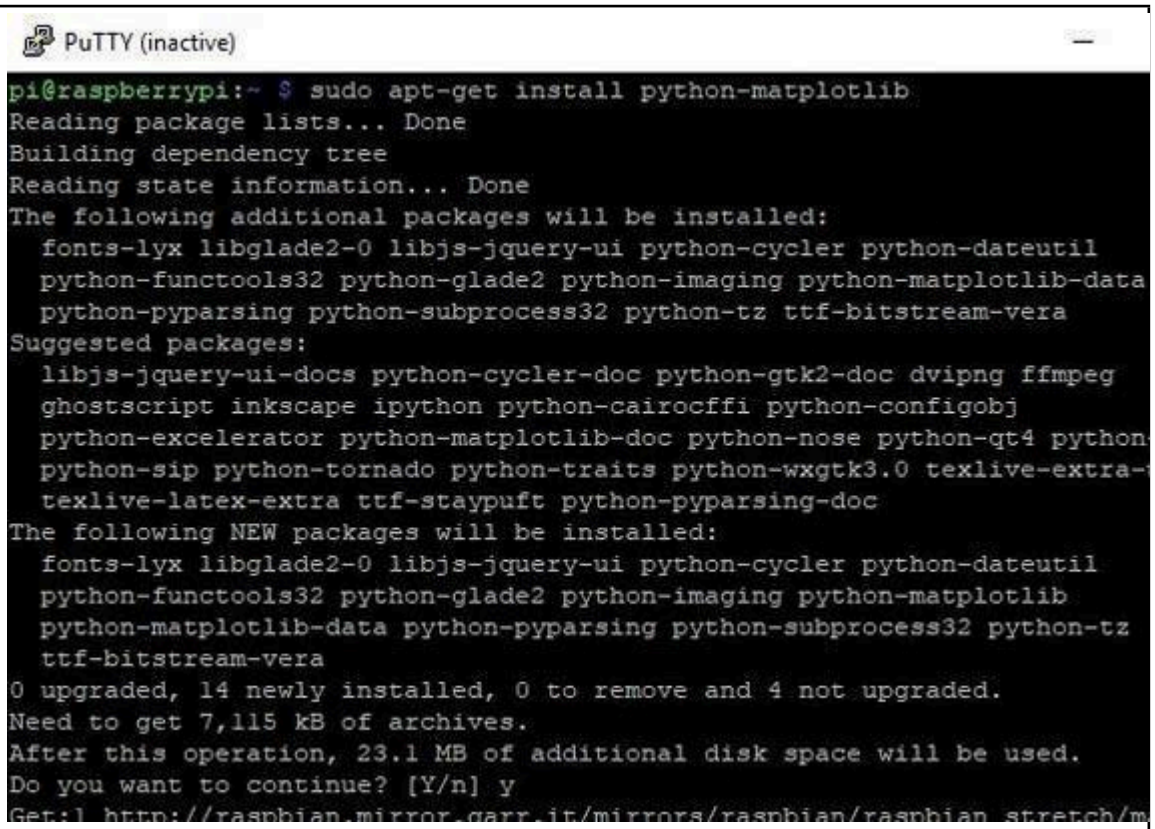
If an error occurs, check to ensure the ADC is well connected to the PI and I2C communication is enabled on the Pi.

Step 5: Install *Matplotlib*

To visualize the data we need to install the *matplotlib* module which is used to plot all kind of graphs in python. This can be done by running;

```
sudo apt-get install python-matplotlib
```

You should see an outcome like the image below.



Step6: Install the *Drawnow* python module

Lastly, we need to install the *drawnow* python module. This module helps us provide live updates to the data plot.

We will be installing *drawnow* via the python package installer; *pip*, so we need to ensure it is installed. This can be done by running;

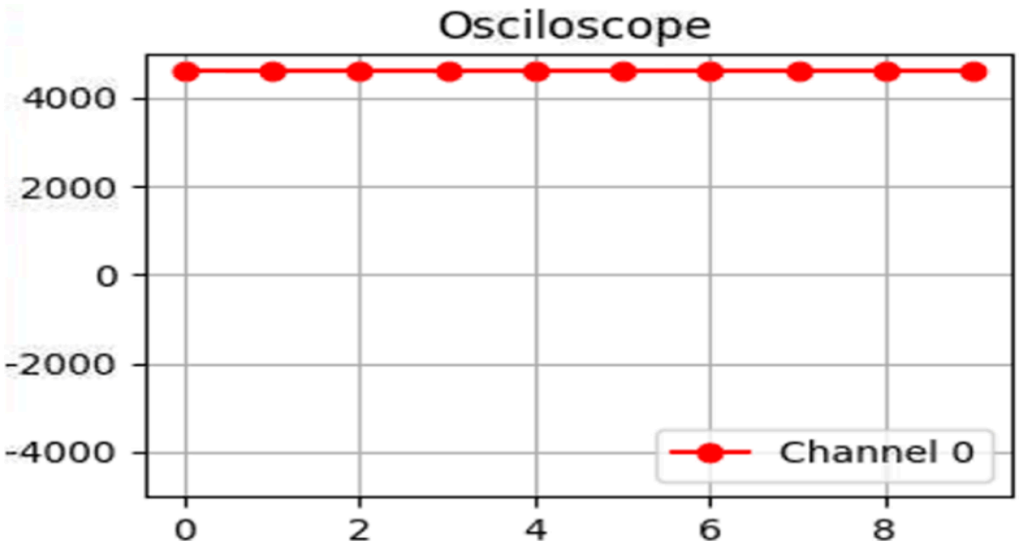
```
sudo apt-get install python-pip
```

We can then use pip to install the *drawnow* package by running:

```
sudo pip install drawnow
```

Download graph code using following github link

$ git clone https://github.com/tyitraspberrypi/Adafruit_Python_ADS1x1z_graph.git



Practical no 5
Raspberry Pi GPS Module Interfacing

Installation Manual

- **GPS** stands for Global Positioning System and used to detect the Latitude and Longitude of any location on the Earth, with exact UTC time (Universal Time Coordinated).
- GPS module is the main component in our vehicle tracking system.
- This device receives the coordinates from the satellite for each and every second, with time and date.
- **GPS module** sends the data related to tracking position in real time, and it sends so many data in NMEA format.
- NMEA format consists of sentence starts from **$GPGGA,** the coordinates, time and other useful information.
- **GPGGA** is referred to **Global Positioning System Fix Data**.
- GPGGA string contains following co-ordinates separated by commas.

Sr No	Identifier	Description	Sr No	Identifier	Description
1	$GPGGA	Global Positioning system fix data	7	FQ	Fix Quality Data
2	HHMMSS.SSS	Time in hour minute seconds and milliseconds format.	8	NOS	No. of Satellites being Used
3	Latitude	Latitude (Coordinate)	9	HPD	Horizontal Dilution of Precision
4	N	Direction N=North, S=South	10	Altitude	Altitude from sea level
5	Longitude	Longitude(Coordinate)	11	M	Meter
6	E	Direction E= East, W=West	12	Height	Height
			13	Checksum	Checksum Data

Hardware Requirements	Software Requirements
• Raspberry Pi Model B/B+ , SD Card • Ethernet Cable / Wi-Fi • Power Supply to Pi • Neo 6m v2 GPS Module • Jumper wires • Breadboard • 2x16 LCD Display • Potentiometer (to adjust Contrast)	• Raspbian Stretch OS • Adafruit Python Char LCD Library

Neo 6m v2 GPS Module

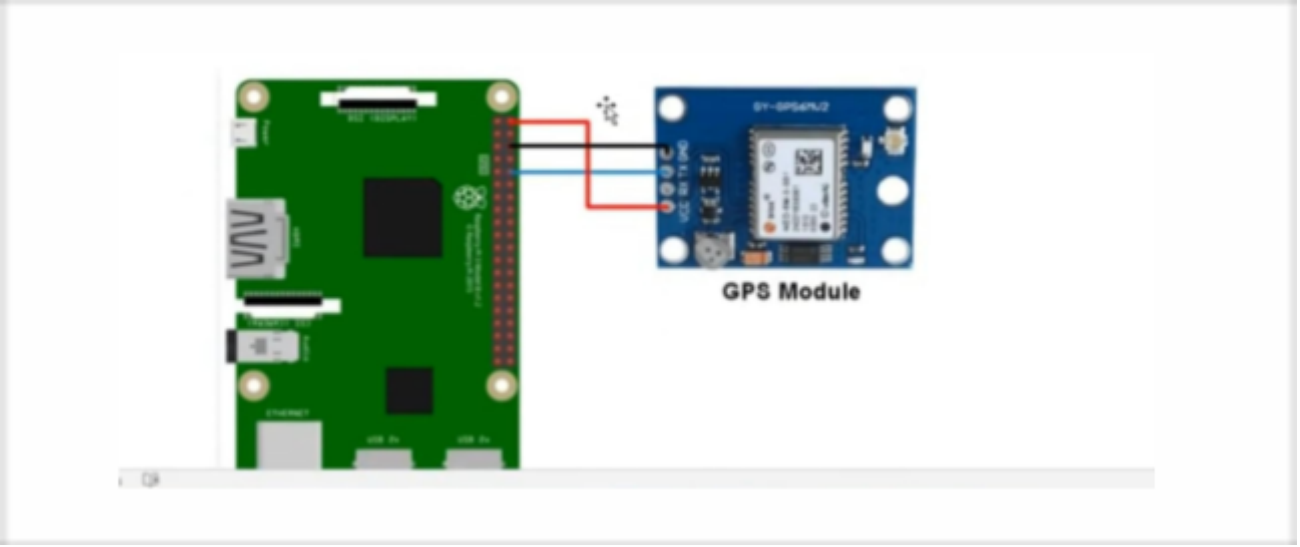
- This boar
module with ante
compatible with
designed to work
- EEPROM
data when powere
- Power Supply R
- Default Baud Ra



Connect GPS Module to RPi

Connection is very simple. Requires 4 Female to Female Jumper Wires

Neo 6m V2 GPS Board Pin	Details	Raspberry Pi Physical Pin	Raspberry Pi Function
VCC	Power	Pin 2	3.3V Power
GND	Common Ground	Pin 6	GND
TXD	Data Output	Pin 10	(UART_RXD0) GPIO15
RXD	Data Input	Not required	(UART_TXD0) GPIO14



Step 1: Update Raspberry Pi



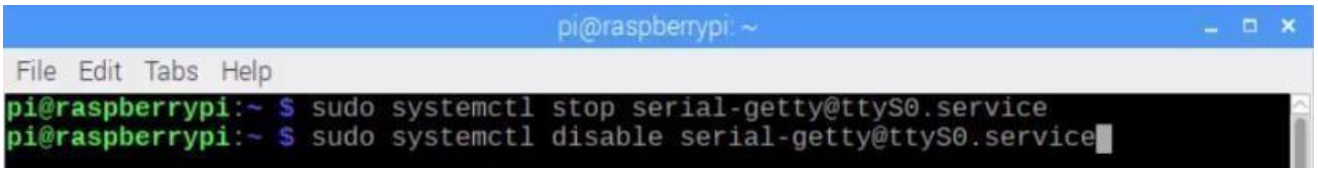
Step 2: edit the /boot/config.txt file



#config starts- do not modify other configuration lines
dtparam=spi=on
dtoverlay=pi3-disable
-bt core freq=250
enable_uart=1
force_turbo-1
#config ends To save and
#exit editor press Ctrl+O> Enter -> ctrl +X

Step 3 **: Reboot Raspberry Pi using the command *sudo reboot***

Step 4 **: Stop and disable the Pi's serial ttyS0 service**



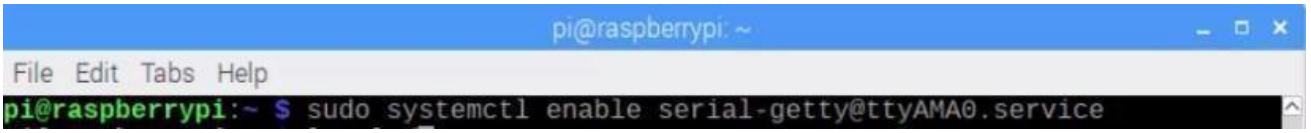
The following commands can be used to enable it again if needed

```
sudo systemctl enable serial-getty@ttyS0.service

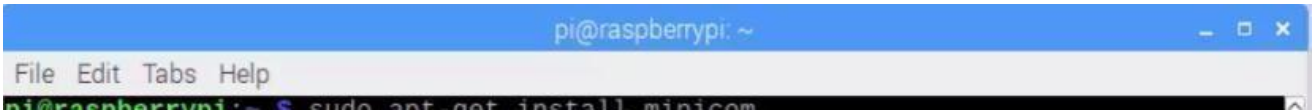
sudo systemctl start serial-getty@ttyS0.service
```

Step 5 **: Reboot Raspberry Pi using the command *sudo reboot***

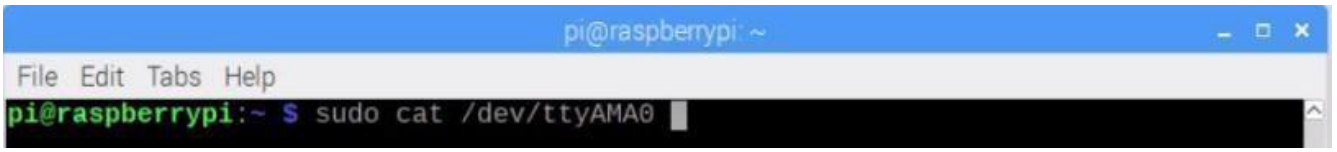
Step 6 **: Now, Enable the ttyAMA0 service**

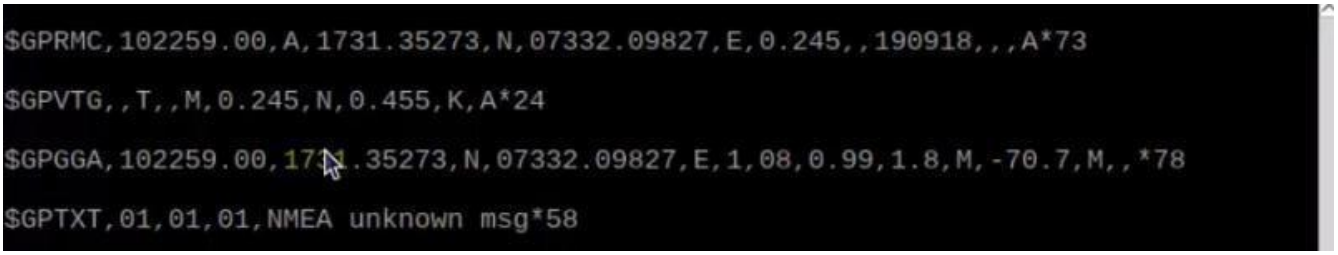


Step 7:Install minicom

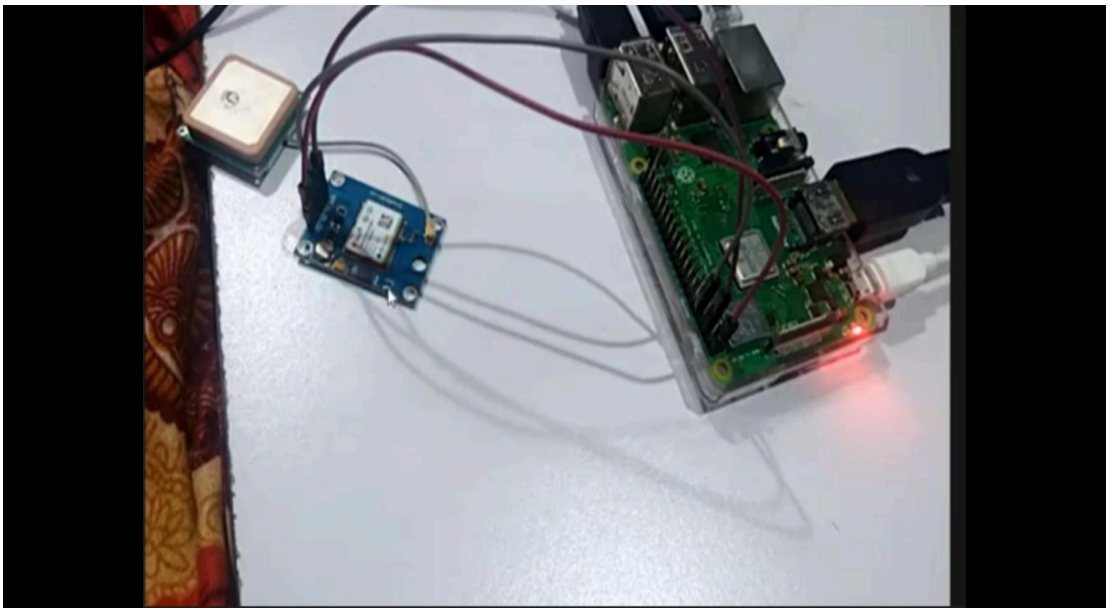


Now Run file by below command





This sentence gives you Latitude found after two commas and Longitude found after four commas.



Practical no 6
Raspberry Pi GPS Module Interfacing

Interfacing Raspberry Pi with RFID.

Installation Manual

RFID stands for **Radio Frequency Identification** uses radio frequency to read information stored in a RFID card or tag. Each card has a unique ID and this makes it a perfect choice for many authentication applications. The RFID authentication systems are easy to design and are cheap in cost. Interfacing RFID Reader with Raspberry Pi can be very useful as you can implement a wide range of applications like:

- Access Control
- Authentication
- e-Ticket ,e-Payment ,e-Toll
- Attendance System

Hardware Requirements

1. Raspberry Pi Model 3 B/B+
2. RFID Reader (RC 522)
3. RFID Tags or Cards
4. Jumper wires (Female to Male)
5. Breadboard

Here, I am using **RFID Reader RC 522**



This module came with two different styles of header pins, one of which needed to be soldered onto the PCB.

The RFID RC522 is a very low-cost RFID (Radio-frequency identification) reader and writer that is based on the MFRC522 microcontroller. This microcontroller provides its data through the SPI protocol and works by creating a 13.56MHz electromagnetic field that it uses to communicate with the RFID tags.

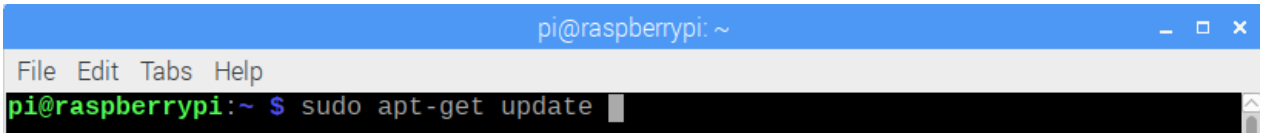
- **Software Requirements**

1. Raspbian Stretch OS
2. SPI Supporting Libraries
3. RC522 Python Library

Connect your RFID reader with Raspberry Pi's GPIO Pins.

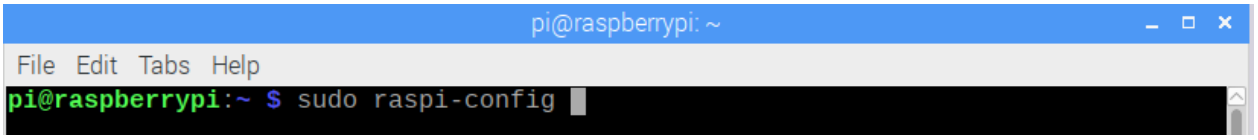
RFID Reader Function Board Pin	RPI Physical Pin	Raspberry
SDA	**24**	**GPIO8 (SPI_CE0_N)**
SCK	**23**	**GPIO11 (SPI0-_CLK)**
MOSI	**19**	**GPIO10 (SPI0_MOSI)**
MISO	**21**	**GPIO9 (SPI0_MISO)**
IRQ	**UNUSED**	
GND	**6**	**GND**
RST	**22**	**GPIO25 (GPIO_GEN6)**
3.3V	**1**	**3.3V PWR**

Step 1: Update Raspberry Pi





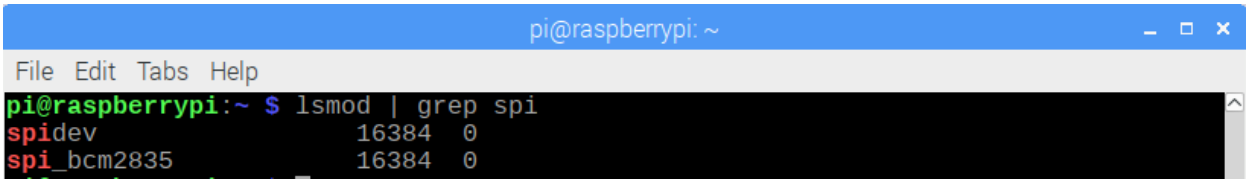
Step 2: Enable SPI Interface using sudo raspi-config tool



Select using Arrow Keys Interfacing Options SPI Enable

Step 3: Reboot Raspberry Pi

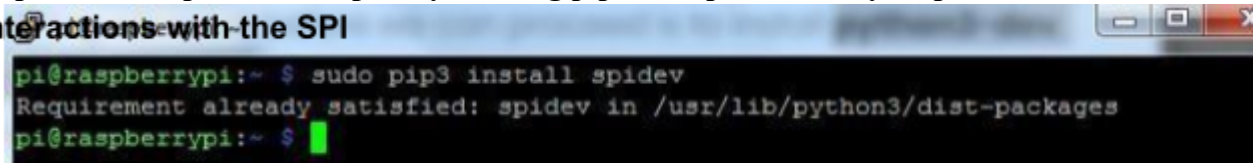
Step 4: Check to make sure that SPI has been enabled.



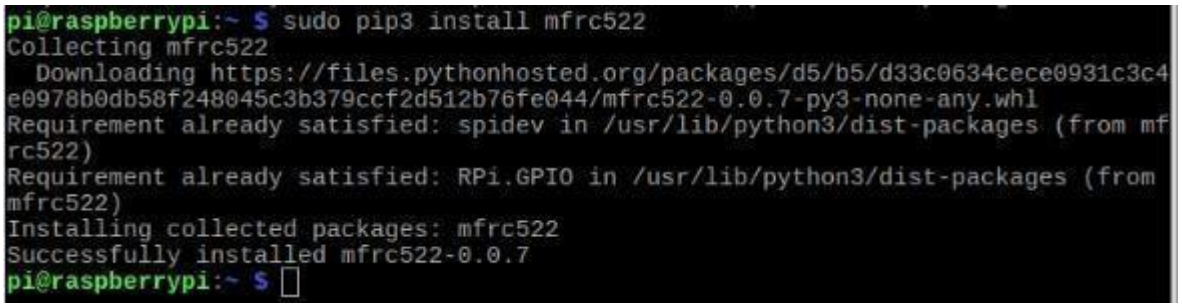
Step 5: Install python3-dev, python3-pip packages to setting up RFID reader



Step 6: Install spidev to Raspberry Pi using pip. The spidev library helps to handle interactions with the SPI

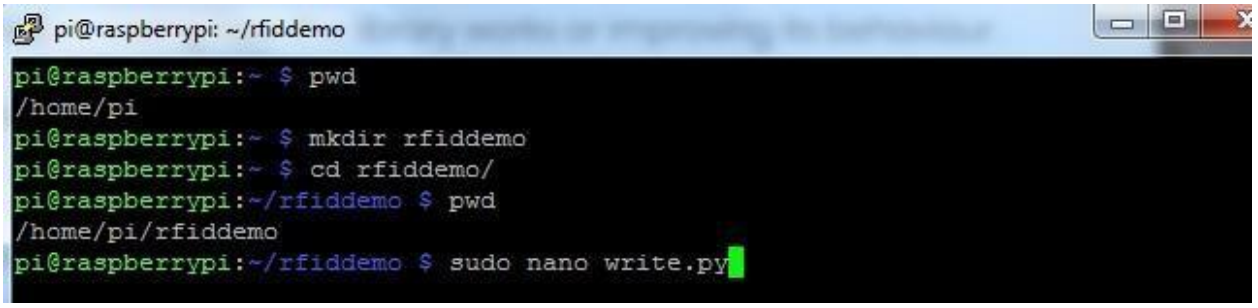


Step 7: Install the MFRC522 library using pip that helps talk to the RC522 module over the SPI interface



There are two files included in this repository:
MFRC522.py **which is an implementation of the RFID RC522 circuit.**
SimpleMFRC522.py **that takes the** MFRC522.py **file and greatly simplifies it.**

Step 8: Write Python script which is used to write data from the RC522 to your RFID tags.




```
import RP1.GPIO as GPIO
from mfrc522 import SimpleMFRC522

reader = SimpleMFRC522()

#welcome message
print("Looking for cards")
print("Press Ctrl+c to STOP")

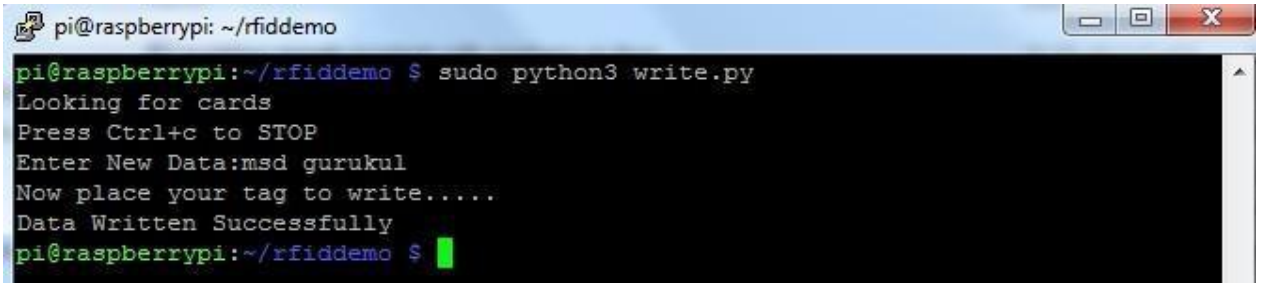
try:
    text=input('Enter New Data:')
    print("Now place your tag to write

    ")
    reader.write(text)
    print("Data Written Successfully    ")
finally:
    GPIO.cleanup()
```

#Write this code in write.py

And Run this file

Output:

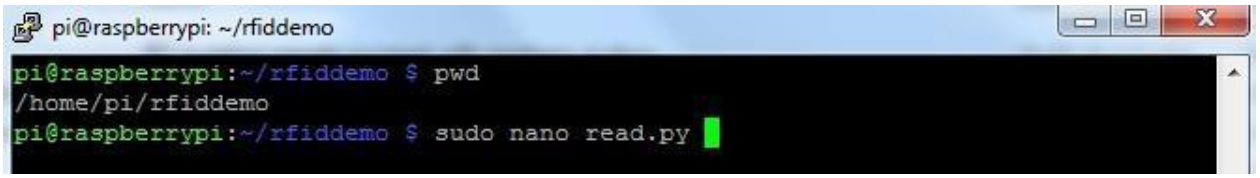


Run above script

When you run script, it asked to write in the new data, in my case I am going to just type in msd gurukul. **Press** Enter **when you are happy with what you have written.**

With that done, simply place your RFID Tag on top of your RFID RC522 circuit. As soon as it detects it, it will immediately write the new data to the tag. You should see "Data Written successfully**" appear in your command line if it was successful.**

Step 8: Write Python script which is used to read this data back off the RFID tag.




```
import RPi.GPIO as GPIO
from mfrc522 import
SimpleMFRC522

reader = SimpleMFRC522()

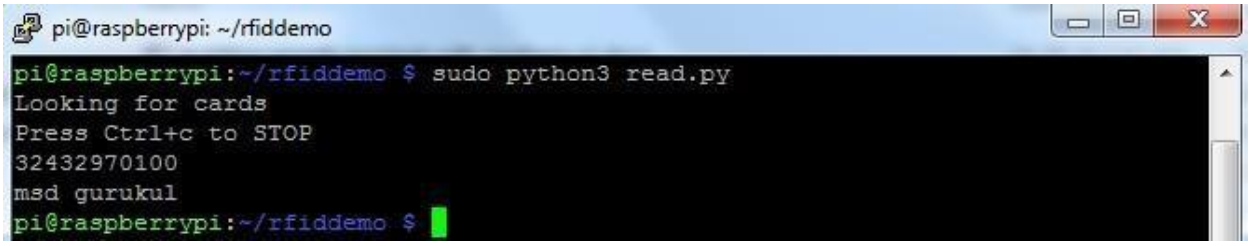
#welcome message
print ("Looking for cards") print
("Press Ctrl+c to STOP")

try:
    id, text = reader.read() print(id)
    print(text) finally:
    GPIO.cleanup ()
```

#Write this code in read.py

And Run this file

Output:



Run above script

Practical NO 7

Interface a 16×2 LCD module with Raspberry l
user-defined messages on the screen.

REQUIRMENT:

Raspberry Pi 400

LCD 1602 with I2C module

4 Female-to-Female jumper wires

🔌 Wiring (I2C LCD)

LCD Pin	Connect to Raspberry Pi GPIO
GND	Pin 6 (GND)
VCC	Pin 4 (5V)
SDA	Pin 3 (GPIO2 - SDA)
SCL	Pin 5 (GPIO3 - SCL)

Diagram:

LCD Pi 400 GPIO
---- ----------
GND --> GND (Pin 6)
VCC --> 5V (Pin 4)
SDA --> SDA (GPIO 2 / Pin 3)
SCL --> SCL (GPIO 3 / Pin 5)

🖥️ Step 1: Enable I2C on Raspberry Pi
sudo raspi-config

Go to Interface Options → I2C → Enable

Reboot your Pi

📦 **Step 2: Install Required Packages**
sudo apt update
sudo apt install i2c-tools python3-smbus python3-pip
pip3 install RPLCD

Check if the LCD is detected:

i2cdetect -y 1

You'll see something like

0 1 2 3 4 5 6 7 8 9 A B C D E F
00: -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- 27 -- --

Note down the address (usually 0x27 or 0x3F).

🧪 Step 3: Test LCD with Python

CODE:

```
from RPLCD.i2c import CharLCD
from time import sleep

lcd = CharLCD('PCF8574', 0x27)  # Change 0x27 if needed

lcd.write_string("Hello from Pi 400!")
sleep(3)
lcd.clear()
```

Save this as lcd_test.py and run:

python3 lcd_test.py

✅ Your LCD should now display the message!



setting up wirelss access point using raspberry pi 400

Practical No 8

setting up wirelss access point using raspberry pi 400

What You'll Need

Raspberry Pi 400 with Raspberry Pi OS (Bookworm or Bullseye)

Internet connection (via Ethernet or USB dongle)

Built-in Wi-Fi (Pi 400 has it)

Terminal access (keyboard, mouse, or SSH)

⚙️ Step-by-Step Setup

🪜 Step 1. Update System

```
sudo apt update
sudo apt upgrade -y
```

🪜 Step 2. Install Required Packages

You'll need hostapd (for access point) and dnsmasq (for DHCP server).

```
sudo apt install hostapd dnsmasq -y
```

Then disable them temporarily:

```
sudo systemctl stop hostapd
sudo systemctl stop dnsmasq
```

🪜 Step 3. Configure Static IP for WLAN

Edit the network configuration:

```
sudo nano /etc/dhcpcd.conf
```

Add these lines at the bottom:

```
interface wlan0
    static ip_address=192.168.4.1/24
    nohook wpa_supplicant
```

Save → Ctrl + O, then Ctrl + X.

Reboot dhcpcd:

sudo service dhcpcd restart

🪜 Step 4. Configure dnsmasq (DHCP)

Backup default file:

sudo mv /etc/dnsmasq.conf /etc/dnsmasq.conf.orig

Create a new one:

sudo nano /etc/dnsmasq.conf

Paste:

interface=wlan0
dhcp-range=192.168.4.2,192.168.4.20,255.255.255.0
,24h

Save and exit.

🪜 Step 5. Configure hostapd (Access Point)

Create config file:

sudo nano /etc/hostapd/hostapd.conf

Paste:

interface=wlan0
driver=nl80211
ssid=Pi400_Hotspot
hw_mode=g
channel=7
wmm_enabled=0
auth_algs=1
ignore_broadcast_ssid=0
wpa=2
wpa_passphrase=raspberry123
wpa_key_mgmt=WPA-PSK
rsn_pairwise=CCMP

Now point hostapd to this config:

sudo nano /etc/default/hostapd

Find the line starting with #DAEMON_CONF and change to:

DAEMON_CONF="/etc/hostapd/hostapd.conf"

🪜 Step 6. Enable and Start Services

sudo systemctl unmask hostapd
sudo systemctl enable hostapd
sudo systemctl enable dnsmasq
sudo systemctl start hostapd
sudo systemctl start dnsmasq

Your Pi is now broadcasting a Wi-Fi network named Pi400_Hotspot ✨
(Password: raspberry123)